Tutorial 10

Paths and spanning trees

Homework

1. Dijkstra's Algorithm: Select any two graphs from Q1 (Parts 1–8) and Q3 (Parts 1–7). For each selected graph, choose one of the specified starting and ending vertex pairs and provide a detailed solution.
2. Critical Path Scheduling: Select any one question from Q4–Q9. Follow the instructions for the selected question and provide a detailed solution.
3. Minimum Spanning Trees: Select any one graph from Q12 (Parts 1–5). Follow the instructions and provide a detailed solution.

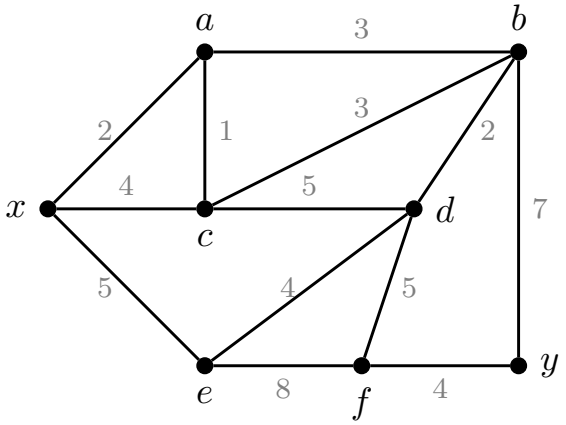
Question 1. Apply Dijkstra's Algorithm to each of the graphs below, using the specified starting and ending vertices.

1.

(a) start x, end y

(b) start a, end y

(c) start a, end e

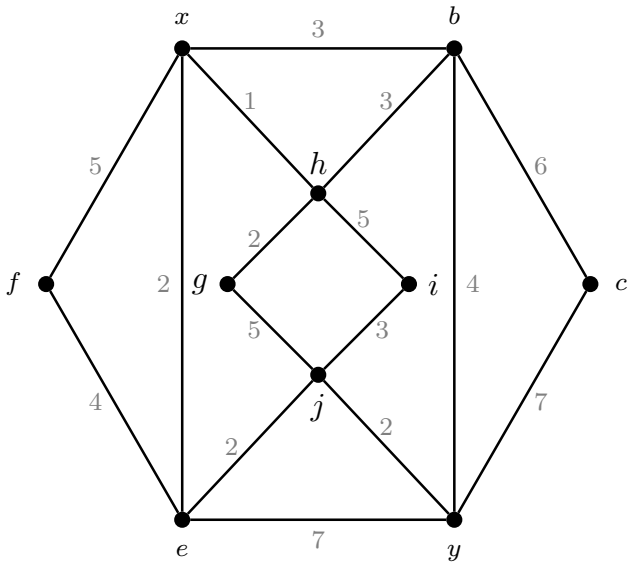


2.

(a) start x, end y

(b) start b, end y

(c) start f, end i

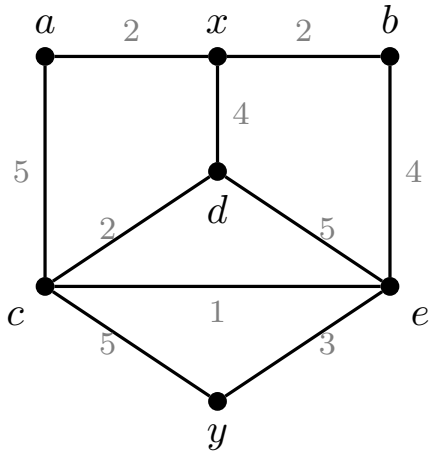


3.

(a) start x, end y

(b) start a, end e

(c) start b, end c

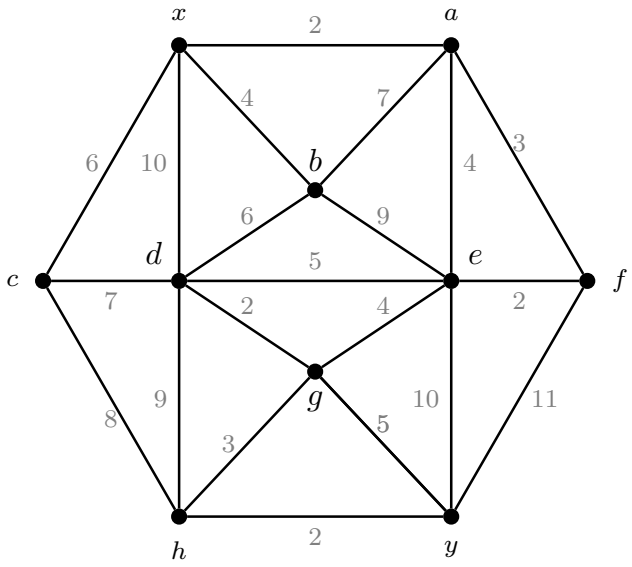


4.

(a) start x, end y

(b) start a, end h

(c) start b, end y

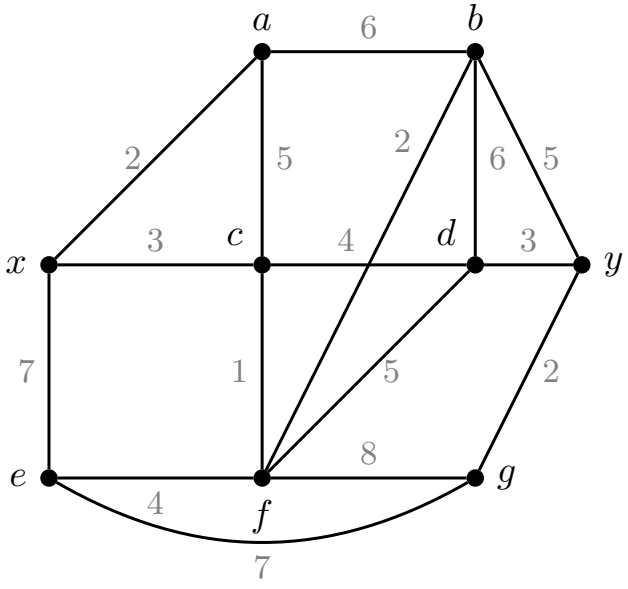


5.

(a) start x, end y

(b) start a, end g

(c) start b, end e

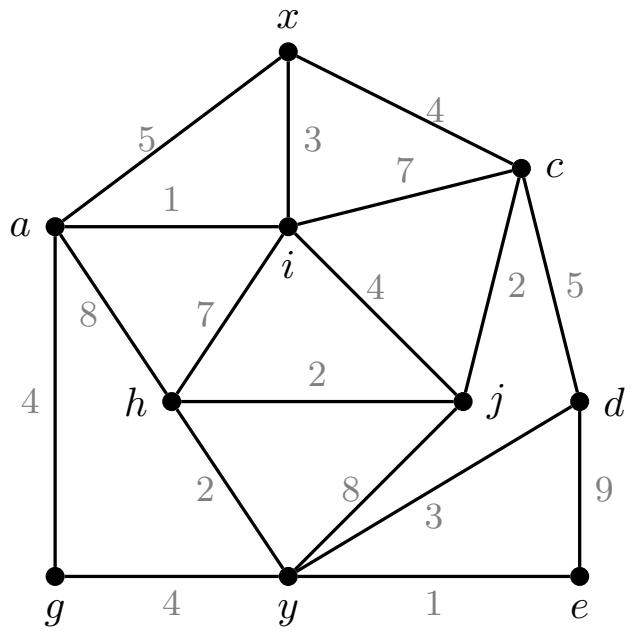


6.

(a) start x, end y

(b) start a, end d

(c) start g, end c

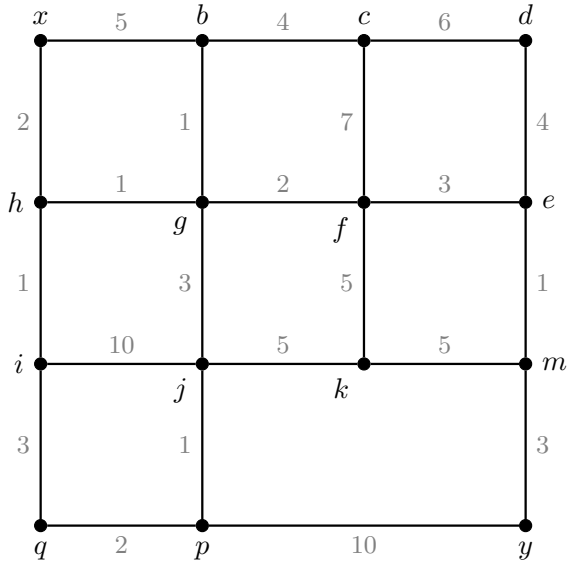


7.

(a) start x, end y

(b) start c, end p

(c) start d, end q

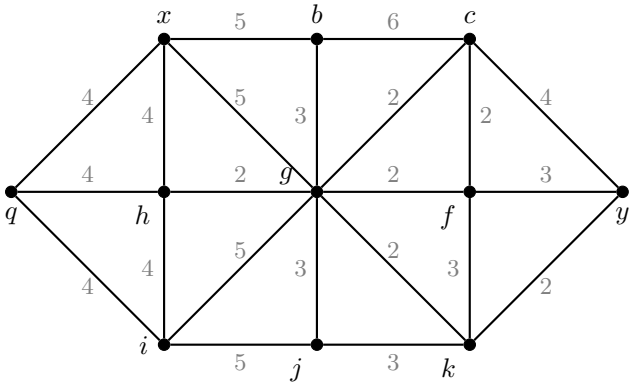


8.

(a) start x, end y

(b) start c, end q

(c) start i, end y



Solution.

1. (a) path: $xaby$, weight: 12
   (b) path: aby, weight: 10
   (c) path: axe, weight: 7
2. (a) path: $xejy$, weight: 6
   (b) path: by, weight: 4
   (c) path: $feji$, weight: 9
3. (a) path: $xbey$, weight: 9
   (b) path: ace, weight: 6
   (c) path: bec, weight: 5
4. (a) path: $xaegy$, weight: 15
   (b) path: $aegh$, weight: 11
   (c) path: $bdgy$, weight: 13
5. (a) path: $xcdy$, weight: 10
   (b) path: $abyg$, weight: 13
   (c) path: bfe, weight: 6
6. (a) path: $xcjhy$, weight: 10
   (b) path: $agyd$, weight: 11
   (c) path: $gyhjc$, weight: 10
7. (a) path: $xhgfemy$, weight: 12

(b) path: $cbgjp$, weight: 9

(c) path: $defghiq$, weight: 14

8. (a) path: $xgky$, weight: 9

(b) path: $cghq$, weight: 8

(c) path: $igky$, weight: 9

Question 2. For a connected, undirected, weighted graph with nonnegative edge weights, modify Dijkstra's Algorithm so that the output consists of the shortest paths from a specified starting vertex to all other vertices in the graph.

Solution. Instead of terminating when the specified ending vertex is highlighted, we continue running Dijkstra's algorithm until all vertices in the graph have been highlighted.

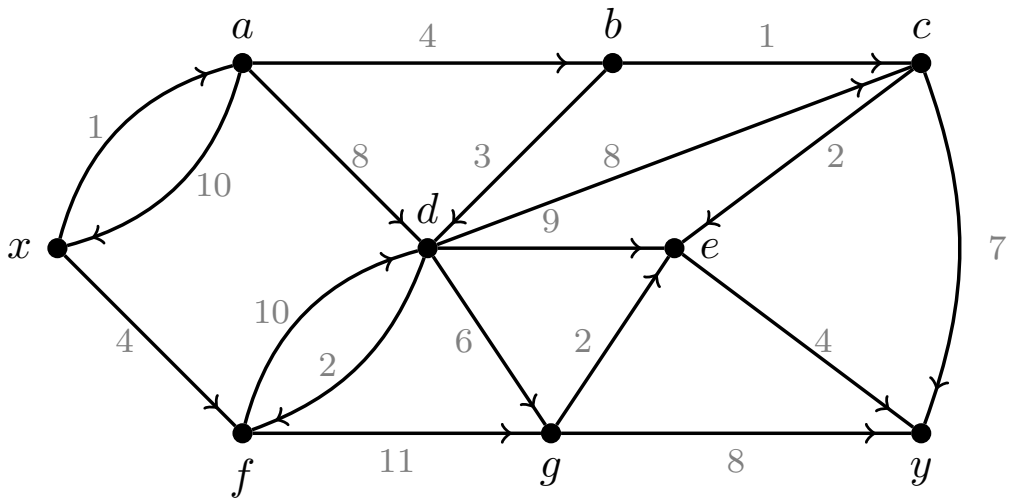
After the algorithm completes, the shortest path from the starting vertex to any other vertex v can be reconstructed by tracing backward from v using the predecessor information stored in the first component of each vertex's label. The total weight of this path is given by the second component of v's label.

Question 3. Apply Dijkstra's Algorithm to each of the directed graphs below, using the specified starting and ending vertices.

1. (a) start x, end y

(b) start a, end y

(c) start f, end c

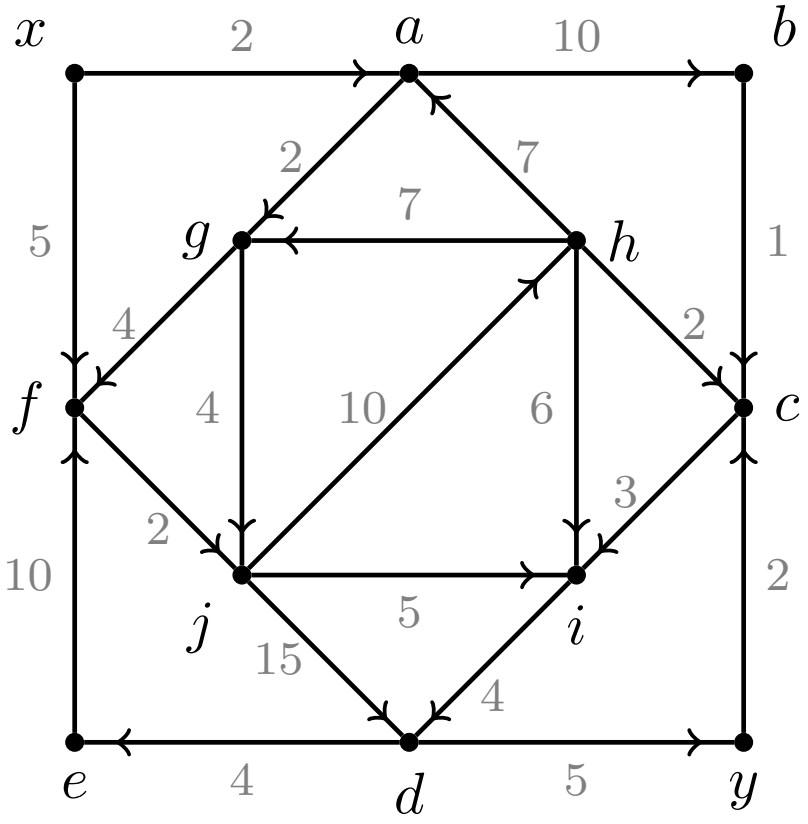


2. (a) start x, end y

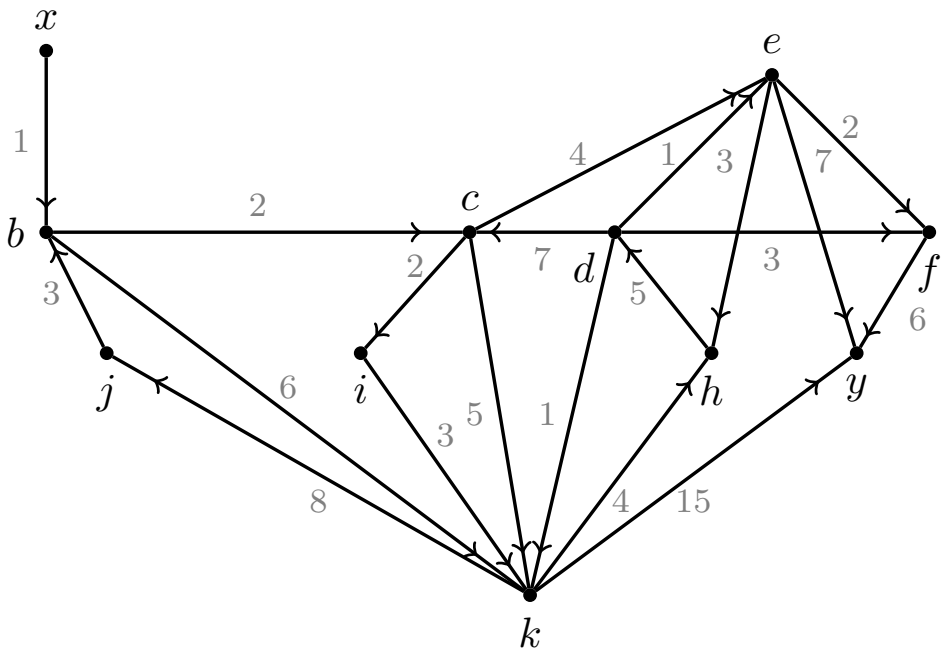
(b) start a, end y

(c) start e, end h

(d) start y, end f

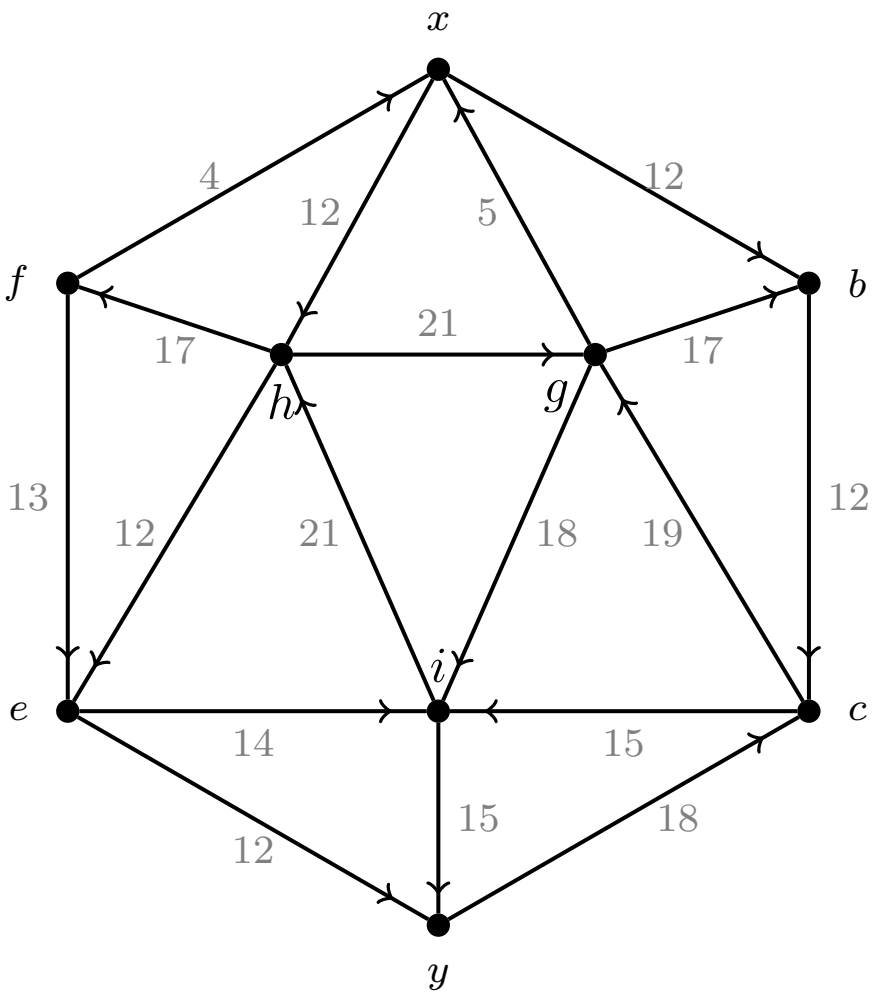


3. (a) start x, end y
   (b) start x, end h
   (c) start b, end f
   (d) start e, end i

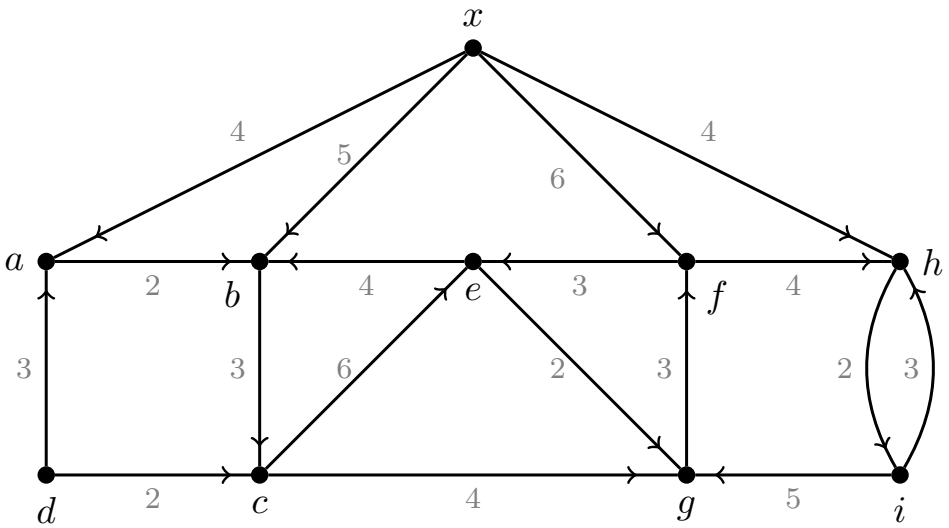


4. (a) start x, end y
   (b) start e, end b
   (c) start f, end c

(d) start f, end y



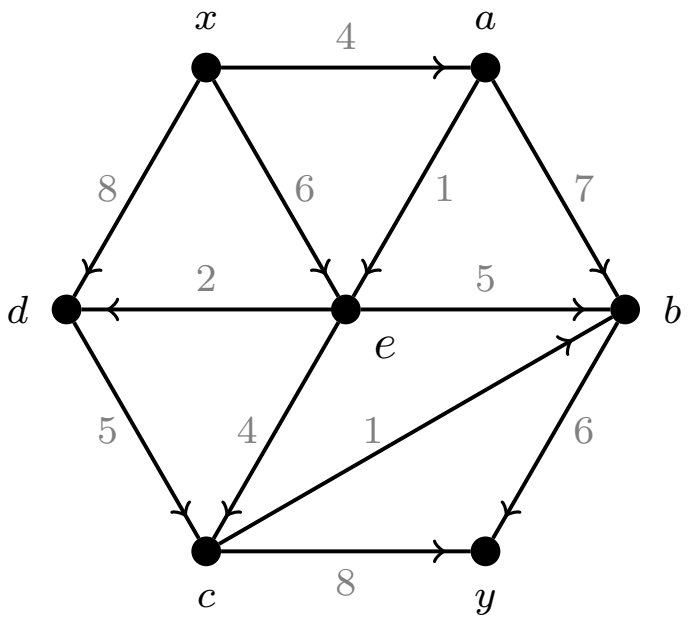
5. (a) start x, end i
   (b) start d, end h
   (c) start a, end g
   (d) start h, end c



6. (a) start x, end y

(b) start d, end y

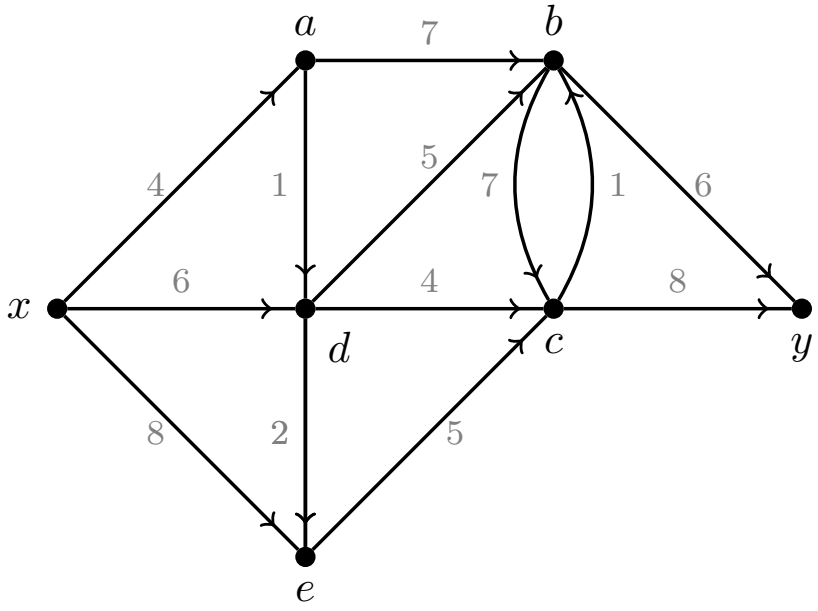
(c) start x, end b



7. (a) start x, end y

(b) start x, end b

(c) start x, end c



Solution.

1. (a) path: $x \to a \to b \to c \to y$, weight: 13

(b) path: $a \to b \to c \to y$, weight: 12

(c) path: $f \to g \to e \to c$, weight: 15

2. (a) path: $x \to f \to j \to i \to d \to y$, weight: 21

(b) path: $a \to g \to j \to i \to d \to y$, weight: 20

(c) path: $e \to f \to j \to h$, weight: 22

(d) path: $y \to c \to i \to d \to e \to f$, weight: 23

3. (a) path: $x \to b \to c \to e \to y$, weight: 14

(b) path: $x \to b \to c \to e \to h$, weight: 10

(c) path: $b \to c \to e \to f$, weight: 8

(d) path: $e \to h \to d \to c \to i$, weight: 17

4. (a) path: $x \to h \to e \to y$, weight: 36

(b) path: $e \to y \to c \to g \to b$, weight: 66

(c) path: $f \to x \to b \to c$, weight: 28

(d) path: $f \to e \to y$, weight: 25

5. (a) path: $x \to h \to i$, weight: 6

(b) path: $d \to c \to g \to f \to h$, weight: 13

(c) path: $a \to b \to c \to g$, weight: 9

(d) path: $h \to i \to g \to f \to e \to b \to c$, weight: 20

6. (a) path: $x \to a \to e \to b \to y$, weight: 16

(b) path: $d \to c \to b \to y$, weight: 12

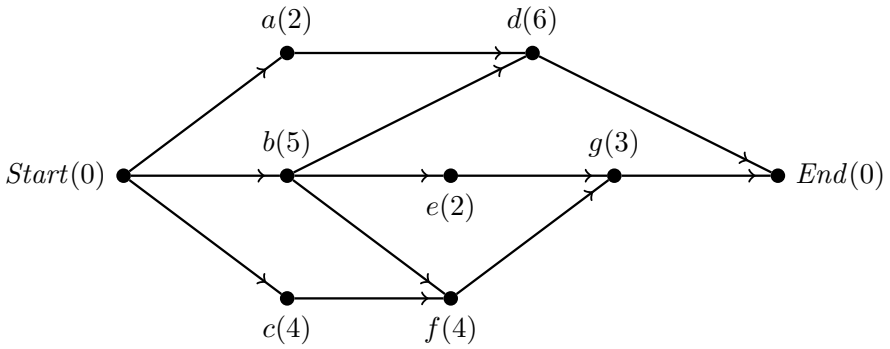
(c) path: $x \to a \to e \to b$, weight: 10

7. (a) path: $x \to a \to d \to b \to y$, weight: 16

(b) path: $x \to a \to d \to b$, weight: 10

(c) path: $x \to a \to d \to c$, weight: 9

Question 4. Consider the project digraph shown below.



1. Use the Critical Path Algorithm to find a schedule with 2 processors.
2. Determine if the schedule is optimal. If not, find a better schedule using 2 processors.

Solution.

1. Critical time

$$\begin{aligned}
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[End] = 3 + 0 = 3,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[g] = 4 + 3 = 7,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[End] = 6 + 0 = 6,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[g] = 2 + 3 = 5,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[f] = 4 + 7 = 11,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[f] = 5 + 7 = 12,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[d] = 2 + 6 = 8,\\
&\mathrm{ct}[Start] = \mathrm{pt}(Start) + \mathrm{ct}[b] = 0 + 12 = 12
\end{aligned}$$

   Critical path: $Start \to b \to f \to g \to End$.

   Critical path priority list: $b - c - a - f - d - e - g$

   Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P_1	b	b	b	b	b	f	f	f	f	e	e	g	g	g
P_2	c	c	c	c	a	a	d	d	d	d	d	d	$*$	$*$

2.

$$OPT \geq \mathrm{ct}[Start] \Longrightarrow OPT \geq 12.$$

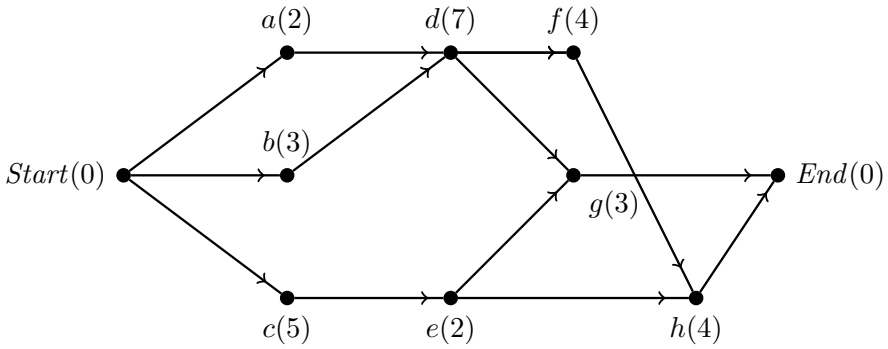
   For 2 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{26}{2} = 13.$$

   The schedule is not optimal. An optimal schedule using 2 processors has a finishing time of 13. An optimal schedule is as follows

	1	2	3	4	5	6	7	8	9	10	11	12	13
P_1	a	a	c	c	c	c	f	f	f	f	g	g	g
P_2	b	b	b	b	b	e	e	d	d	d	d	d	d

Question 5. Consider the project digraph shown below.



1. Use the Critical Path Algorithm to find a schedule with 2 processors.
2. Determine if the schedule is optimal. If not, find a better schedule using 2 processors.

Solution.

1. Critical time

$$\begin{aligned}
&\mathrm{ct}[h] = \mathrm{pt}(h) + \mathrm{ct}[End] = 4 + 0 = 4,\\
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[End] = 3 + 0 = 3,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[h] = 4 + 4 = 8,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[h] = 2 + 4 = 6,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[f] = 7 + 8 = 15,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[e] = 5 + 6 = 11,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[d] = 3 + 15 = 18,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[d] = 2 + 15 = 17,\\
&\mathrm{ct}[Start] = \mathrm{pt}(Start) + \mathrm{ct}[b] = 0 + 18 = 18
\end{aligned}$$

Critical path: $Start \to b \to d \to f \to h \to End$.

Critical path priority list: $b - a - d - c - f - e - h - g$

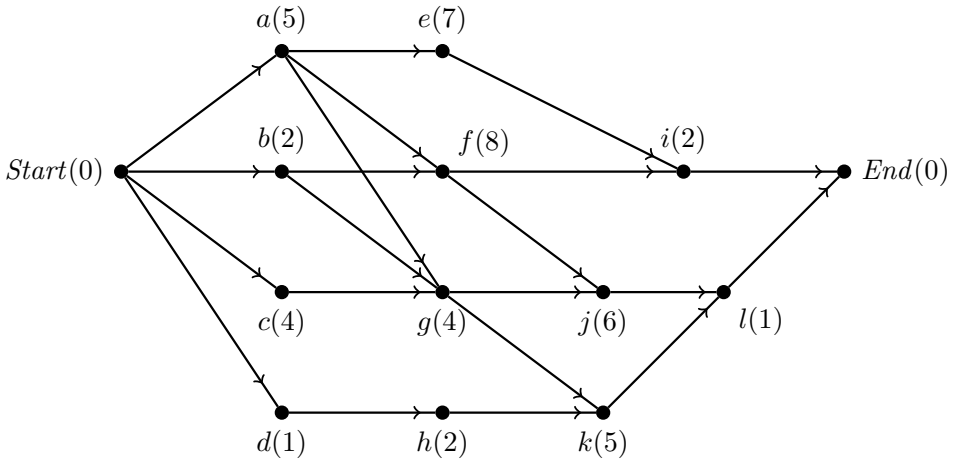
Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P_1	b	b	b	d	d	d	d	d	d	d	f	f	f	f	h	h	h	h
P_2	a	a	c	c	c	c	c	e	e	$*$	g	g	g	$*$	$*$	$*$	$*$	$*$

$$OPT \geq \mathrm{ct}[Start] \Longrightarrow OPT \geq 18.$$

The schedule is optimal since it has finishing time 18.

Question 6. Consider the project digraph shown below.



1. Use the Critical Path Algorithm to find a schedule with 2 processors.
2. Use the Critical Path Algorithm to find a schedule with 3 processors.
3. Using the critical-path and total-processing-time lower bounds introduced in the lecture, determine whether the optimality of either schedule can be established. Justify your conclusions.

Solution.

1. Critical time

$$\begin{aligned}
&\mathrm{ct}[l] = \mathrm{pt}(l) + \mathrm{ct}[End] = 1 + 0 = 1,\\
&\mathrm{ct}[k] = \mathrm{pt}(k) + \mathrm{ct}[l] = 5 + 1 = 6,\\
&\mathrm{ct}[j] = \mathrm{pt}(j) + \mathrm{ct}[l] = 6 + 1 = 7,\\
&\mathrm{ct}[i] = \mathrm{pt}(i) + \mathrm{ct}[End] = 2 + 0 = 2,\\
&\mathrm{ct}[h] = \mathrm{pt}(h) + \mathrm{ct}[k] = 2 + 6 = 8,\\
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[j] = 4 + 7 = 11,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[j] = 8 + 7 = 15,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[i] = 7 + 2 = 9,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[h] = 1 + 8 = 9,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[g] = 4 + 11 = 15,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[f] = 2 + 15 = 17,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[f] = 5 + 15 = 20,\\
&\mathrm{ct}[Start] = \mathrm{pt}(start) + \mathrm{ct}[a] = 0 + 20 = 20
\end{aligned}$$

Critical path: $Start \to a \to f \to j \to l \to End$.

Critical path priority list: $a - b - f - c - g - e - d - h - j - k - i - l$

Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
P_1	a	a	a	a	a	f	f	f	f	f	f	f	f	d	h	h	j	j	j	j	j	j	i	i
P_2	b	b	c	c	c	c	g	g	g	g	e	e	e	e	e	e	e	k	k	k	k	k	l	$*$

2. Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P_1	a	a	a	a	a	f	f	f	f	f	f	f	f	j	j	j	j	j	j	l
P_2	b	b	d	h	h	g	g	g	g	k	k	k	k	k	$*$	$*$	$*$	$*$	$*$	$*$
P_3	c	c	c	c	$*$	e	e	e	e	e	e	e	$*$	i	i	$*$	$*$	$*$	$*$	$*$

3. For 2 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{47}{2} = 23.5 \Longrightarrow OPT \geq 24$$

Thus, the schedule in part 1 is optimal.

For 3 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{47}{3} \Longrightarrow OPT \geq 16$$

We also know that

$$OPT \geq \text{ct}[Start] \Longrightarrow OPT \geq 20.$$

Thus, the schedule in part 2 is optimal.

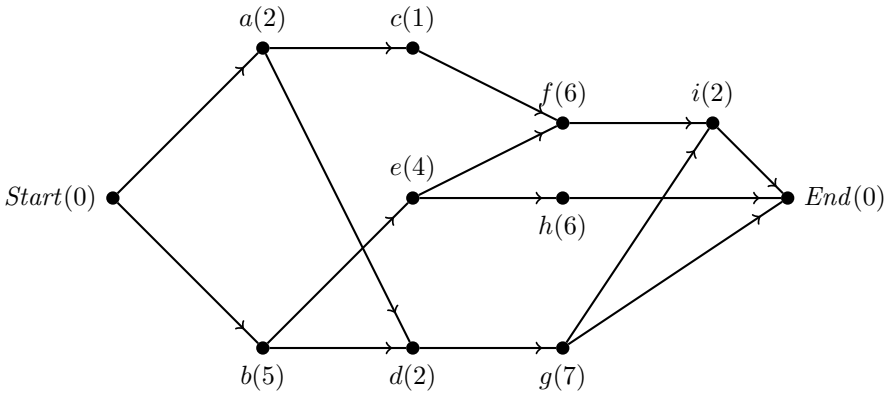
Question 7. The table below lists 9 tasks that comprise a project, as well as their processing times and precedence relationships.

Task	**Processing Time**	**Precedence Relationships**
a	2	
b	5	
c	1	a
d	2	a, b
e	4	b
f	6	c, e
g	7	d
h	6	e
i	2	f, g

1. Draw the project digraph.
2. Use the Critical Path Algorithm to find a schedule with 2 processors.
3. Use the Critical Path Algorithm to find a schedule with 3 processors.
4. Using the critical-path and total-processing-time lower bounds introduced in the lecture, determine whether the optimality of either schedule can be established. Justify your conclusions.

Solution.

1. Project digraph



2. Critical time

$$\begin{aligned}
&\mathrm{ct}[i] = \mathrm{pt}(i) + \mathrm{ct}[End] = 2 + 0 = 2,\\
&\mathrm{ct}[h] = \mathrm{pt}(h) + \mathrm{ct}[End] = 6 + 0 = 6,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[i] = 6 + 2 = 8,\\
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[i] = 7 + 2 = 9,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[f] = 4 + 8 = 12,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[g] = 2 + 9 = 11,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[f] = 1 + 8 = 9,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[e] = 5 + 12 = 17,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[d] = 2 + 11 = 13,\\
&\mathrm{ct}[Start] = \mathrm{pt}(start) + \mathrm{ct}[b] = 0 + 17 = 17
\end{aligned}$$

Critical path: $Start \to b \to e \to f \to i \to End$.

Critical path priority list: $b - a - e - d - c - g - f - h - i$

Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P_1	b	b	b	b	b	e	e	e	e	f	f	f	f	f	f	i	i	$*$	$*$	$*$
P_2	a	a	c	$*$	$*$	d	d	g	g	g	g	g	g	g	h	h	h	h	h	h

3. Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
P_1	b	b	b	b	b	e	e	e	e	f	f	f	f	f	f	i	i
P_2	a	a	c	$*$	$*$	d	d	g	g	g	g	g	g	g	$*$	$*$	$*$
P_3	$*$	$*$	$*$	$*$	$*$	$*$	$*$	$*$	$*$	h	h	h	h	h	h	$*$	$*$

4. For 2 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{35}{2} = 17.5 \Longrightarrow OPT \geq 18$$

We also know that

$$OPT \geq \mathrm{ct}[Start] \Longrightarrow OPT \geq 17.$$

The two-processor schedule has a finishing time of 20, while the lower bounds give $OPT \geq 18$. Therefore, these bounds alone do not establish whether the schedule is optimal.

The three-processor schedule has a finishing time of 17, which equals the critical-path lower bound. Hence, the three-processor schedule is optimal.

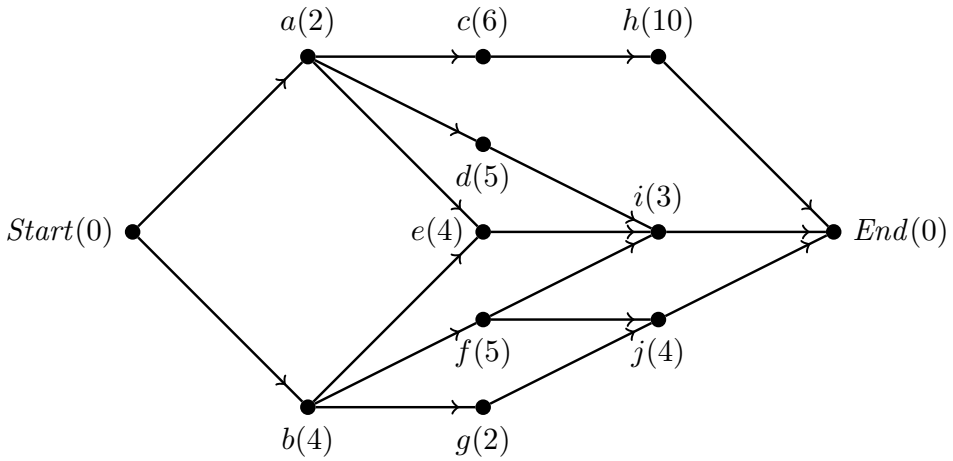
Question 8. The table below lists 10 tasks that comprise a project, as well as their processing times and precedence relationships.

Task	Processing Time	Precedence Relationships
a	2	
b	4	
c	6	a
d	5	a
e	4	a, b
f	5	b
g	2	b
h	10	c
i	3	d, e, f
j	4	f, g

1. Draw the project digraph.
2. Use the Critical Path Algorithm to find a schedule with 2 processors.
3. Use the Critical Path Algorithm to find a schedule with 3 processors.
4. Using the critical-path and total-processing-time lower bounds introduced in the lecture, determine whether the optimality of either schedule can be established. Justify your conclusions.

Solution.

1. Project digraph



2. Critical time

$$\begin{aligned}
&\mathrm{ct}[j] = \mathrm{pt}(j) + \mathrm{ct}[End] = 4 + 0 = 4,\\
&\mathrm{ct}[i] = \mathrm{pt}(i) + \mathrm{ct}[End] = 3 + 0 = 3,\\
&\mathrm{ct}[h] = \mathrm{pt}(h) + \mathrm{ct}[End] = 10 + 0 = 10,\\
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[j] = 2 + 4 = 6,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[j] = 5 + 4 = 9,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[i] = 4 + 3 = 7,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[i] = 5 + 3 = 8,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[h] = 6 + 10 = 16,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[f] = 4 + 9 = 13,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[c] = 2 + 16 = 18,\\
&\mathrm{ct}[Start] = \mathrm{pt}(start) + \mathrm{ct}[a] = 0 + 18 = 18
\end{aligned}$$

Critical path: $Start \to a \to c \to h \to End$.

Critical path priority list: $a - c - b - h - f - d - e - g - j - i$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
P_1	a	a	c	c	c	c	c	c	h	h	h	h	h	h	h	h	h	h	g	g	j	j	j	j
P_2	b	b	b	b	f	f	f	f	f	d	d	d	d	d	e	e	e	e	i	i	i	$*$	$*$	$*$

3. Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P_1	a	a	c	c	c	c	c	c	h	h	h	h	h	h	h	h	h	h
P_2	b	b	b	b	f	f	f	f	f	g	g	j	j	j	j	$*$	$*$	$*$
P_3	$*$	$*$	d	d	d	d	d	e	e	e	e	i	i	i	$*$	$*$	$*$	$*$

4. For 2 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{45}{2} = 22.5 \Longrightarrow OPT \geq 23$$

We also know that

$$OPT \geq \mathrm{ct}[Start] \Longrightarrow OPT \geq 18.$$

The two-processor schedule has a finishing time of 24, while the lower bounds give $OPT \geq 23$. Therefore, the bounds do not establish whether this schedule is optimal.

The three-processor schedule has a finishing time of 18, which equals the critical-path lower bound. Hence, this schedule is optimal.

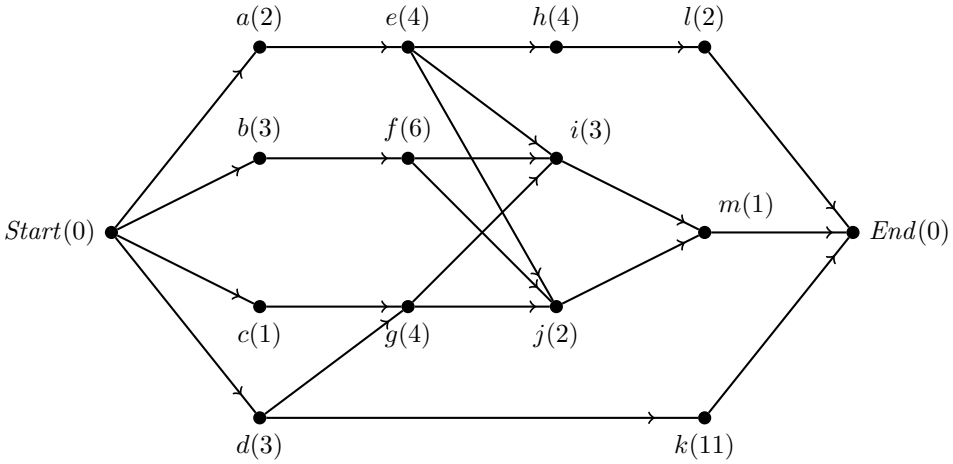
Question 9. The table below lists 13 tasks that comprise a project, as well as their processing times and precedence relationships.

Task	Processing Time	Precedence Relationships
a	2	
b	3	
c	1	
d	3	
e	4	a
f	6	b
g	4	c, d
h	4	e
i	3	e, f, g
j	2	e, f, g
k	11	d
l	2	h
m	1	i, j

1. Draw the project digraph.
2. Use the Critical Path Algorithm to find a schedule with 2 processors.
3. Use the Critical Path Algorithm to find a schedule with 3 processors.
4. Using the critical-path and total-processing-time lower bounds introduced in the lecture, determine whether the optimality of either schedule can be established. Justify your conclusions.

Solution.

1. Project digraph



2. Critical time

$$\begin{aligned}
&\mathrm{ct}[m] = \mathrm{pt}(m) + \mathrm{ct}[End] = 1 + 0 = 1,\\
&\mathrm{ct}[l] = \mathrm{pt}(l) + \mathrm{ct}[End] = 2 + 0 = 2,\\
&\mathrm{ct}[j] = \mathrm{pt}(j) + \mathrm{ct}[m] = 2 + 1 = 3,\\
&\mathrm{ct}[i] = \mathrm{pt}(i) + \mathrm{ct}[m] = 3 + 1 = 4,\\
&\mathrm{ct}[h] = \mathrm{pt}(h) + \mathrm{ct}[l] = 4 + 2 = 6,\\
&\mathrm{ct}[k] = \mathrm{pt}(k) + \mathrm{ct}[End] = 11 + 0 = 11,\\
&\mathrm{ct}[g] = \mathrm{pt}(g) + \mathrm{ct}[i] = 4 + 4 = 8,\\
&\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[i] = 6 + 4 = 10,\\
&\mathrm{ct}[e] = \mathrm{pt}(e) + \mathrm{ct}[h] = 4 + 6 = 10,\\
&\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[k] = 3 + 11 = 14,\\
&\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[g] = 1 + 8 = 9,\\
&\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[f] = 3 + 10 = 13,\\
&\mathrm{ct}[a] = \mathrm{pt}(a) + \mathrm{ct}[e] = 2 + 10 = 12,\\
&\mathrm{ct}[Start] = \mathrm{pt}(start) + \mathrm{ct}[d] = 0 + 14 = 14
\end{aligned}$$

Critical path: $Start \to d \to k \to End$.

Critical path priority list: $d - b - a - k - e - f - c - g - h - i - j - l - m$

Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
P_1	d	d	d	a	a	e	e	e	e	f	f	f	f	f	f	g	g	g	g	i	i	i	m
P_2	b	b	b	k	k	k	k	k	k	k	k	k	k	k	c	h	h	h	h	j	j	l	l

3. Schedule

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P_1	d	d	d	k	k	k	k	k	k	k	k	k	k	k	l	l
P_2	b	b	b	f	f	f	f	f	f	h	h	h	h	j	j	$*$
P_3	a	a	e	e	e	e	c	g	g	g	g	i	i	i	$*$	m

4. For 2 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{46}{2} = 23$$

Thus the schedule with 2 processors is optimal.

For 3 processors, we have

$$OPT \geq \frac{\text{sum of processing times}}{\text{number of processors}} \Longrightarrow OPT \geq \frac{46}{3} \Longrightarrow OPT \geq 16$$

Thus the schedule with 3 processors is optimal.

Question 10.

For each of the graphs described below, determine if G is (i) definitely a tree, (ii) definitely not a tree, or (iii) may or may not be a tree. Explain your answer or demonstrate with a proper graph.

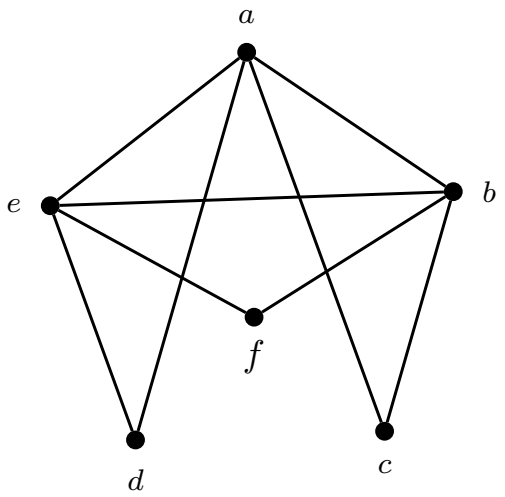
1. G has 10 vertices and 11 edges.
2. G has 10 vertices and 9 edges.
3. G is connected and every vertex has degree 1 or 2.
4. There is exactly one path between any two vertices of G.
5. G is connected with 15 vertices and 14 edges.
6. G is connected with 15 vertices and 20 edges.
7. G has two components, each with 9 vertices and 8 edges.

Solution.

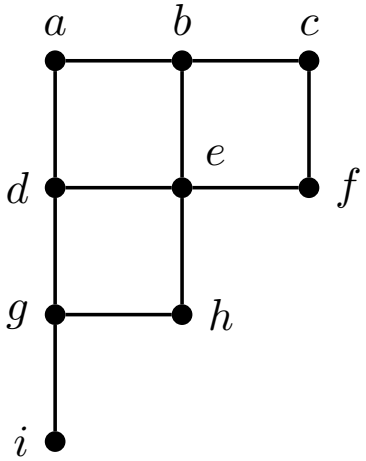
- 1: definitely not a tree, since a tree with 10 vertices must have 9 edges.
- 2: may or may not be a tree, since a graph with 10 vertices and 9 edges could be a tree, but it could also be disconnected.
- 3: may or may not be a tree. A connected graph in which every vertex has degree 1 or 2 can be a path, which is a tree, or a cycle, which is not a tree.
- 4: definitely a tree, since having exactly one path between any two vertices is a defining property of trees.
- 5: definitely a tree, since a connected graph with 15 vertices and 14 edges is a tree.
- 6: definitely not a tree, since a connected graph with 15 vertices and 20 edges has too many edges to be a tree.
- 7: definitely not a tree, since a tree must be connected, but this graph has two components.

Question 11. Find a spanning tree for each of the graphs below.

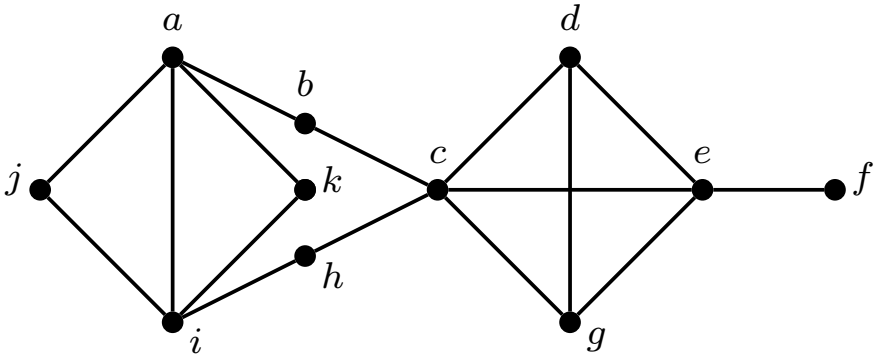
1.



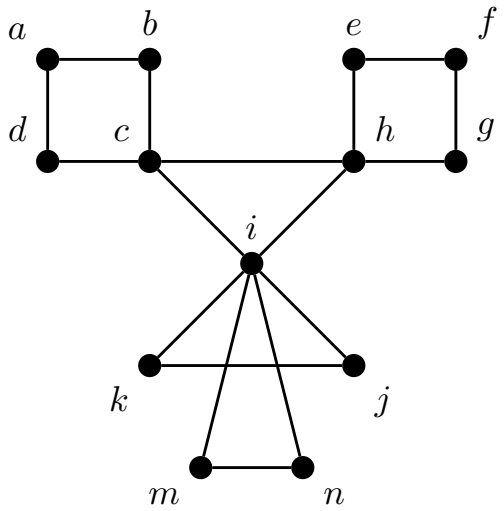
2.



3.

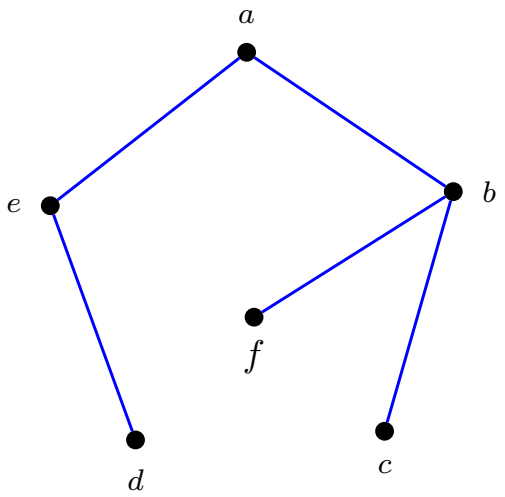


4.

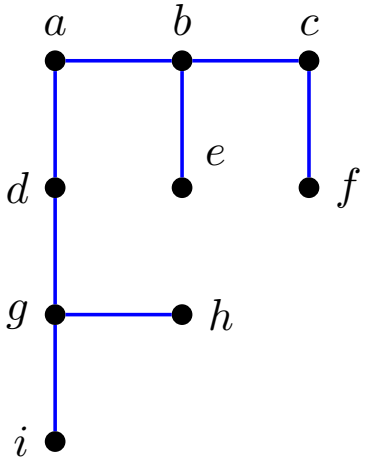


Solution.

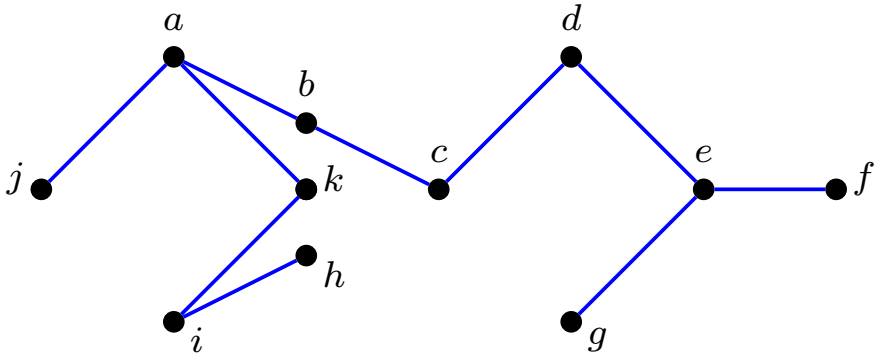
1.



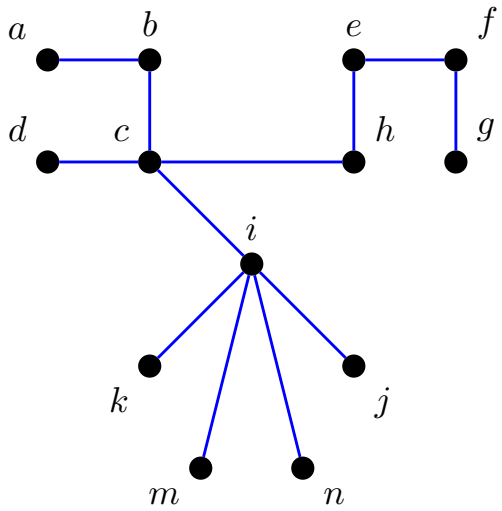
2.



3.



4.

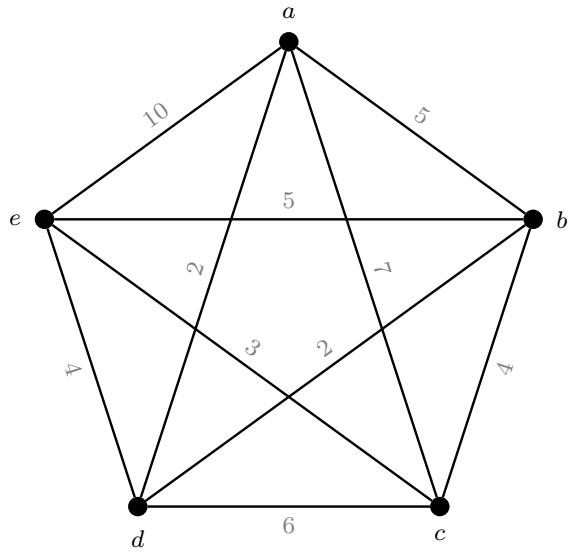


Question 12. Find a minimum spanning tree for each of the graphs below using

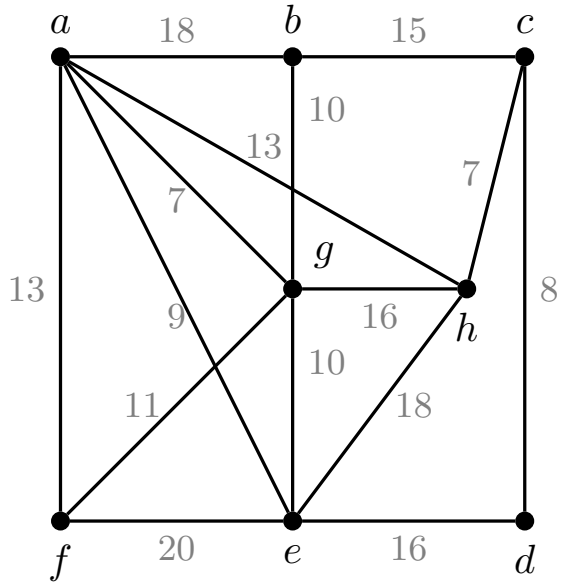
(i) Kruskal's Algorithm

(ii) Prim's Algorithm

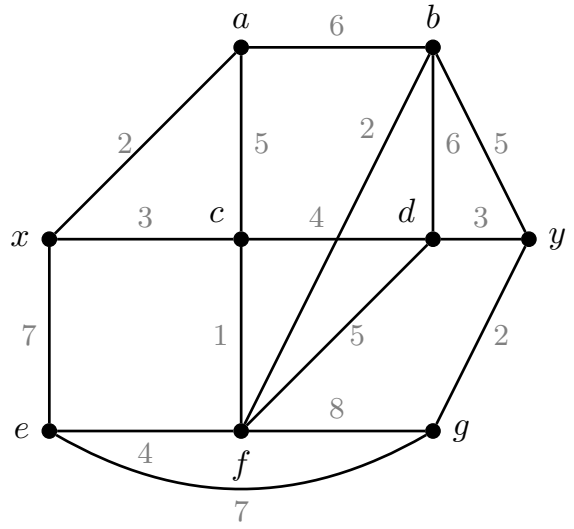
1.



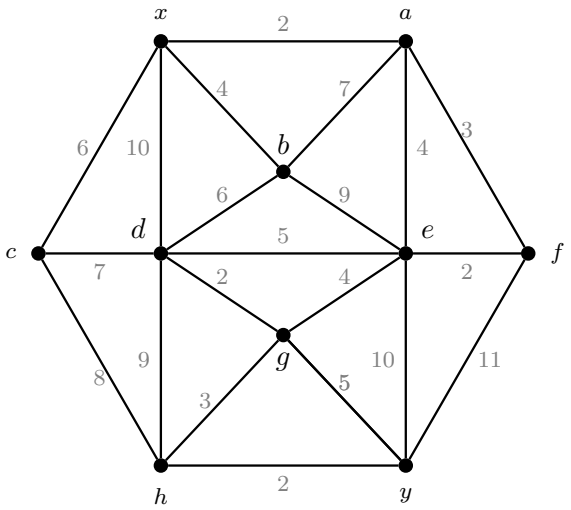
2.



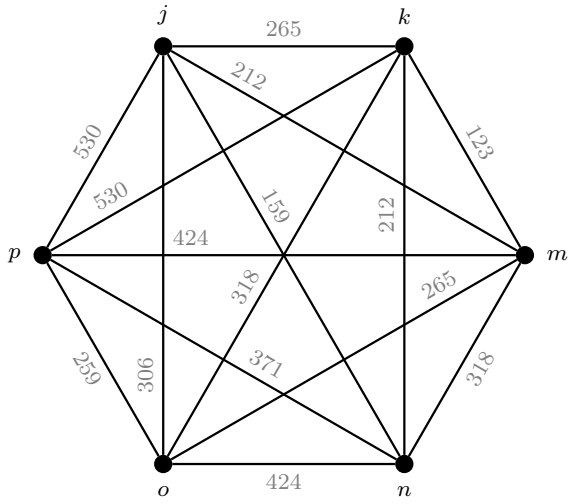
3.



4.



5.



Solution.

1. Weight of MST: 11
   (i) Kruskal's Algorithm: ad, bd, ce, bc
   (ii) Prim's Algorithm: ad, db, bc, ce
2. Weight of MST: 65
   (i) Kruskal's Algorithm: $ag, ch, cd, ae, bg, gf, ah$
   (ii) Prim's Algorithm: $ag, ae, gb, gf, ah, hc, cd$
3. Weight of MST: 21
   (i) Kruskal's Algorithm: $cf, bf, gy, xa, dy, xc, cd, ef$
   (ii) Prim's Algorithm: $ax, xc, cf, bf, cd, dy, yg, fe$
4. Weight of MST: 28
   (i) Kruskal's Algorithm: $dg, ef, hy, xa, af, hg, eg, xb, xc$
   (ii) Prim's Algorithm: $ax, af, fe, eg, gd, gh, hy, xb, xc$
5. Weight of MST: 1018
   (i) Kruskal's Algorithm: km, jn, jm, op, mo
   (ii) Prim's Algorithm: nj, jm, mk, mo, op

Question 13. Find a minimum spanning tree for the graph represented by the table below.

	a	b	c	d	e	f	g
a	$\cdot$	5	7	8	10	3	11
b	5	$\cdot$	2	4	1	12	7
c	7	2	$\cdot$	6	7	5	4
d	8	4	6	$\cdot$	2	10	12
e	10	1	7	2	$\cdot$	6	9
f	3	12	5	10	6	$\cdot$	15
g	11	7	4	12	9	15	$\cdot$

Solution.

- Weight of MST: 17
- Kruskal's Algorithm: be, bc, de, af, cg, ab
- Prim's Algorithm: af, ab, be, bc, de, cg

Question 14. Kruskal's Algorithm and Prim's Algorithm are both written with a connected graph as an input. Determine how each of these would perform if the input was a disconnected graph.

Solution. Kruskal's Algorithm will produce a spanning forest of the disconnected graph; Prim's Algorithm would produce a spanning tree of the connected component containing the chosen root vertex.

Question 15. How would you modify Kruskal's and Prim's algorithm if a specific edge must be included in the spanning tree? Would the resulting tree be a minimum spanning tree? Explain your answer.

Solution. Insert a Step 0 in which the required edge is included in the growing tree or forest. Then continue the algorithm as usual, while avoiding the creation of cycles.

The resulting spanning tree need not be a minimum spanning tree for the original graph, because forcing a particular edge may require us to exclude a lighter edge that would otherwise appear in an MST.

Question 16. The Optos Cable Company is in the process of expanding its fiber optic network over the coming years. While this expansion will require the installation of new cable lines, the company aims to minimize overall costs. Given that the cost per meter of cable is fixed, it is essential to construct the network using the shortest total length of cable possible.

Construction has already begun: the first two phases of the expansion are complete, with cables laid between Quechee and Vinton, and between Vinton and Tempe. However, due to changes in other infrastructure projects, the required cable lengths between certain cities have been updated.

Using the already-installed connections and the revised cable lengths provided below, determine the most efficient way to complete the network expansion. Your goal is to minimize the additional cable length required to connect all cities. Calculate both the additional cable length needed and the total length of the completed network.

	Mesa	Natick	Quechee	Rutland	Tempe	Vinton
Mesa	·	29	32	25	34	45
Natick	29	·	50	46	45	51
Quechee	32	50	·	41	28	19
Rutland	25	46	41	·	40	35
Tempe	34	45	28	40	·	15
Vinton	45	51	19	35	15	·

Solution.

- Already-installed edges: qv and vt, with total weight $19 + 15 = 34$.
- Additional edges required: mr, mn, and mq.
- Additional cable length required: $25 + 29 + 32 = 86$.
- Total weight of the completed network: $34 + 86 = 120$.

- Kruskal's Algorithm: vt, qv, mr, mn, mq.
- Prim's Algorithm: qv, vt, qm, mr, mn.

Question 17. The Reverse Delete Algorithm finds a minimum spanning tree by deleting the largest weighted edges as long as you do not disconnect the graph. In essence, it is Kruskal's Algorithm in reverse. Apply the Reverse Delete Algorithm to the graphs in Question 12.
Solution.

1. Total Weight: 11.
   - MST Edges: bd, ce, de, ad.
   - Deleted Edges: ae, ac, cd, ab, be, bc.
2. Total Weight:65.
   - MST Edges: $ch, ag, ah, cd, fg, bg, ae$.
   - Deleted Edges:$ef, ab, eh, de, gh, bc, af, eg$.
3. Total Weight:21.
   - MST Edges: $ax, cd, cx, ef, gy, dy, cf, bf$.
   - Deleted Edges:$fg, ex, eg, ab, bd, ac, by, df$.
4. Total Weight:28.
   - MST Edges: $gh, ax, bx, af, cx, dg, ef, eg, hy$.
   - Deleted Edges:$fy, dx, ey, be, dh, ch, ab, cd, bd, de, gy, ae$.
5. Total Weight:1018.
   - MST Edges: km, kn, op, jn, mo.
   - Deleted Edges:$jp, kp, mp, no, np, ko, mn, jo, jk, jm$.

Question 18. Under what circumstances would Reverse Delete be a better choice than Kruskal's Algorithm? Under what circumstance would Kruskal's be a better choice? Explain your answer.

Solution. If a graph is very sparse (the number of edges is close to $n-1$), then Reverse Delete would only need a few steps to obtain a tree. However, if a graph is very dense, so that the number of edges is much greater than $n-1$, then Reverse Delete would require many more deletions than Kruskal's Algorithm would require edge selections.

Question 19. We studied two different algorithms for finding a minimum spanning tree. Both Kruskal and Prim cited the work of the Czech mathematician Otakar Borůvka, who is credited with the first minimum spanning tree algorithm from 1926. Below is a description of his algorithm.

Borůvka's Algorithm

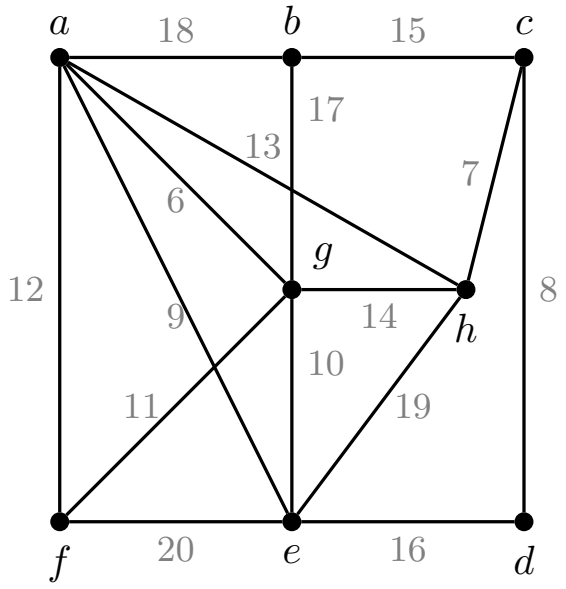
Input: Weighted connected graph $G = (V, E)$ where all the weights are distinct.
Steps:

1. Let T be the forest where each component consists of a single vertex.
2. For each vertex v of G, add the edge of least weight incident to v to T.
3. If T is connected, then it is a minimum spanning tree for G. Otherwise, for each component C of T, find the edge of least weight from a vertex in C to a vertex not in C. Add the edge to T.
4. Repeat Step (3) until T has only one component, making T a tree.

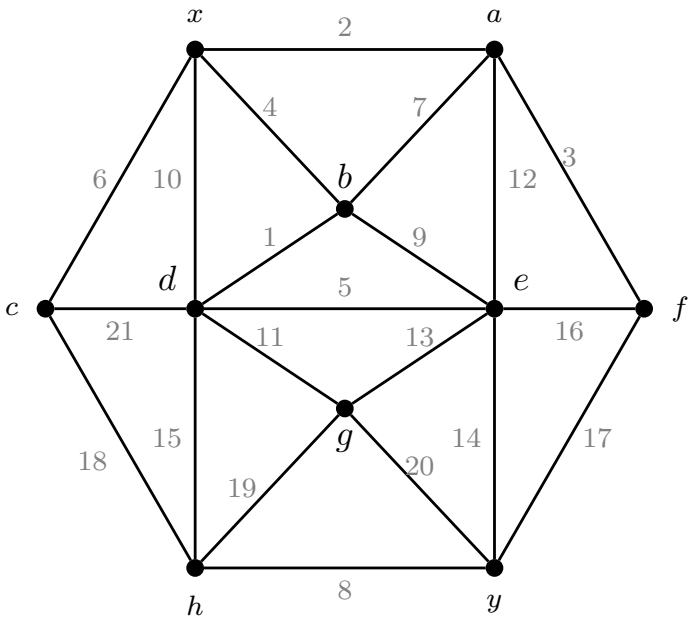
Output: A minimum spanning tree for G.

Apply Borůvka's Algorithm to the following two graphs. Use either Kruskal's Algorithm or Prim's Algorithm to verify that Borůvka's Algorithm found a minimum spanning tree.

1.



2.



Solution.

1. Weight of MST: 69
   - Borůvka's Algorithm: $ag, cb, gf, ae, ch, cd, ah$
   - Kruskal's Algorithm: $ag, ch, cd, ae, gf, ah, cb$
   - Prim's Algorithm: $ag, ae, gf, ah, hc, cd, cb$

2. Weight of MST: 54

   - Borůvka's Algorithm: $xa, bd, xc, af, de, hy, dg, xb, ye$
   - Kruskal's Algorithm: $bd, xa, af, xb, de, xc, hy, dg, ye$
   - Prim's Algorithm: $ax, af, xb, bd, de, xc, dg, ey, yh$

Enrichment Questions

Question 20. You have decided to start a small business that manufactures a product. You are trying to determine the number of employees you will need, as well as how long it will take to produce one item. Write a report where Critical Paths are used to answer these questions. To complete the report, you will need to:

1. Name your business and the product.
2. Determine the different stages of production and their precedence relationships.
3. Draw the project digraph.
4. Include at least 10 different tasks.
5. Ensure at least 2 tasks must rely on multiple tasks.
6. Form schedules using the Critical Path Algorithm using 2, 3, and 4 processors.

Question 21. You are starting a new business that will serve customers in 7 different cities. You need to determine the distance from your hub location to each of your customers.

1. Choose 7 cities from the table on the next page and choose one to be your hub location.
2. Use the modification of Dijkstra's Algorithm from Problem 2 to minimize the mileage from your hub location to each of the customer locations.

City (Acronym)	NYC	LAX	CHI	HOU	PHI	PHX	SAT	SND	DAL	SJC	AUS	JAX	SFO
New York (NYC)	*	2448	712	1419	81	2142	1583	2431	1372	2552	1513	836	2569
Los Angeles (LAX)	2448	*	1744	1372	2391	357	1203	112	1239	306	1226	2146	348
Chicago (CHI)	712	1744	*	943	664	1453	1054	1733	806	1840	981	865	1857
Houston (HOU)	1419	1372	943	*	1341	1015	189	1303	225	1610	146	822	1644
Philadelphia (PHI)	81	2391	664	1341	*	2080	1507	2370	1299	2502	1437	759	2520
Phoenix (PHX)	2142	357	1453	1015	2080	*	848	299	886	615	869	1793	654
San Antonio (SAT)	1583	1203	1054	189	1507	848	*	1128	253	1453	74	1011	1489
San Diego (SND)	2431	112	1733	1303	2370	299	1128	*	1183	417	1156	2090	459
Dallas (DAL)	1372	1239	806	225	1299	886	253	1183	*	1451	182	908	1483
San Jose (SJC)	2552	306	1840	1610	2502	615	1453	417	1451	*	1466	2344	42
Austin (AUS)	1513	1226	981	146	1437	869	74	1156	182	1466	*	960	1502
Jacksonville (JAX)	836	2146	865	822	759	1793	1011	2090	908	2344	960	*	2373
San Francisco (SFO)	2569	348	1857	1644	2520	654	1489	459	1483	42	1502	2373	*

Table 1: Distance matrix between major U.S. cities